A Major Project Report

on

“An Agentic Framework for Real-Time Civic Intelligence”

By

1.Vansh Bohra
2.Disha Jain
3.Drashti Panchal
4.Sakshi Sanap

under the guidance of

Dr. Nilesh Bhelkar

MANJARA CHARITABLE TRUST
RAJIV GANDHI INSTITUTE OF TECHNOLOGY, MUMBAI

Department of Artificial Intelligence and Data Science

University of Mumbai

April – 2026

MANJARA CHARITABLE TRUST

RAJIV GANDHI INSTITUTE OF TECHNOLOGY, MUMBAI

Juhu-Versova Link Road Versova, Andheri(W), Mumbai-53.

Department of Artificial Intelligence and Data Science

Certificate

This is to certify that

1.Vansh Bohra
2.Disha Jain
3.Drashti Panchal
4.Sakshi Sanap

Have satisfactorily completed this project entitled

An Agentic Framework for Real-Time Civic Intelligence

Towards the partial fulfilment of the

BACHELOR OF ENGINEERING
IN
ARTIFICIAL INTELLIGENCE AND DATA SCIENCE

as laid by University of Mumbai.

Dr. Nilesh Bhelkar — **Guide**

Dr. Jyoti Deshmukh — **Head of Department**

Dr. Sanjay Bokade
Principal

Project Report Approval for B. E.

This project report entitled **"An Agentic Framework for Real-Time Civic Intelligence"** by **"Vansh Bohra, Disha Jain, Drashti Panchal, Sakshi Sanap"** is approved for the degree of Bachelor of Engineering in Artificial Intelligence and Data Science.

Examiners

1. _____

2. _____

Date:

Place:

Declaration

We wish to state that the work embodied in this project titled "**An Agentic Framework for Real-Time Civic Intelligence**" forms our own contribution to the work carried out under the guidance of "**Dr.Nilesh Bhelkar**" at the Rajiv Gandhi Institute of Technology.

We declare that this written submission represents our ideas in our own words and where others' ideas or words have been included, we have adequately cited and referenced the original sources. we also declare that we have adhered to all principles of academic honesty and integrity and have not misrepresented or fabricated or falsified any idea/data/fact/source in our submission. we understand that any violation of the above will be cause for disciplinary action by the Institute and can also evoke penal action from the sources which have thus not been properly cited or from whom proper permission has not been taken when needed.

Vansh Bohra (805)

Disha Jain (810)

Drashti Panchal (822)

Sakshi Sanap (826)

Abstract

This project presents a scalable and fault-tolerant news analytics pipeline that integrates real-time streaming and batch processing to deliver comprehensive insights from multiple global and localized news sources. As the modern digital landscape generates vast volumes of unstructured data, the system addresses the critical challenges of high-velocity ingestion and multi-layered analysis by ingesting news articles from seven independent APIs into Apache Kafka, which serves as the central streaming backbone to ensure high-throughput, fault tolerance, and horizontal scalability through topic partitioning. Adopting a dual-layer architecture inspired by the Lambda Architecture, the system comprises a Speed Layer for immediate updates and a Batch Layer for large-scale historical analytics; in the Speed Layer, a dedicated Kafka consumer writes incoming events into MongoDB for low-latency storage, while the Batch Layer utilizes Apache Flink with fixed-time windowing for rigorous data cleaning, enrichment, and normalization. These processed datasets are structured within MinIO following a (Bronze-Silver-Gold) Lakehouse architecture, providing a transparent data lineage from raw ingestion to curated, analytics-ready records. To unify these layers, the system employs FastAPI to expose RESTful endpoints for both real-time and historical data retrieval, which are then visualized through a professional-grade Streamlit web application. This Streamlit frontend serves as a dynamic command center, bridging the gap between immediate feeds and long-term trends to offer intuitive dashboards for sentiment analysis and trend detection. Collectively, this architecture delivers a robust, production-grade solution for news monitoring and decision-making support across industries such as media, finance, and policy research.

Keywords: Apache Kafka, Lambda Architecture, Speed Layer, Batch Layer, MongoDB, MinIO, FastAPI , Streamlit

Contents

List of Figures ii
List of Tables iii
List of Symbols iv
1 Introduction 1
1.1 Introduction Description ---------- 1
1.2 Organization of Report---------- 2
2 Literature Review 3
2.1 Survey Existing system ---------- 3
2.2 Limitation Existing system ---------- 4
2.3 Problem Statement and Objectives ---------- 4
2.3.1 Objectives---------- 4
2.4 Scope ---------- 5
3 Proposed System 6
3.1 Proposed System Statement ---------- 6
3.2 Framework ---------- 6
3.3 Design details---------- 8
3.3.1 System Architecture---------- 8
3.3.2 Detailed Design ---------- 8
3.4 Methodology ---------- 10
3.4.1 Procedures---------- 11
3.4.2 Pseudocode---------- 12
4 Results & Discussions 16
4.1 Software and Hardware Set up---------- 16
4.1.1 Software Requirements ---------- 16
4.1.2 Hardware Requirements ---------- 16
4.2 Results ---------- 17
4.3 Performance Evaluation Parameters ---------- 25
5 Conclusion 26
5.1 Conclusion ---------- 26

References ---------- 27

List of Figures

Fig No.	Figure Name	Page No.
4.1	Docker setup	14
4.2	Speed Producer Logs	14
4.3	Speed Layer Consumer Logs	15
4.4	Kafka Topic - news-events	15
4.5	Kafka Message Data (news-events Topic)	16
4.6	MinIO Object Store	16
4.7	Validation of MinIO Data Layers	17
4.8	Latest news document fetched from Mongo	17
4.9	API JSON response display for analytics	18
4.10	API JSON response display for News endpoint(/search_news)	18
4.11	API JSON response display for News endpoint(/stats)	19
4.12	The Dashboard display	19
4.13	Geospatial visualization of civic events and sentiment distribution (Geo Intelligence dashboard)	20
4.14	Insights dashboard showing historical analytics and article volume trends across pipeline layers	20
4.15	Detailed event view displaying news article content and metadata information	21

List of Tables

Table No.	Table Name	Page No.
1	Technology Stack Of The Civic Intelligence Framework	7
2	Comparative Analysis Of The Proposed Framework With Related Framework	24

List of Symbols

Symbol	Meaning
e	Event (news article)
$f_{kw}(e)$	Keyword-based importance score
$g_{loc}(e)$	Location-based importance score
α	Weight parameter (set to 0.6)
W_n	Time window interval
T	Duration of window (300 seconds)
n	Window index (integer value)
$H(x)$	Fused knowledge score for event
q_i	Reliability weight of source i
$h_i(x)$	Event presence indicator (0 or 1)
i	Index of data source
x	Event instance

Chapter 1

Introduction

1.1 Introduction

The rapid growth of digital news platforms has created a pressing need for intelligent systems capable of delivering real-time insights while simultaneously building a reliable historical record for deeper, long-term analysis. Most conventional architectures are designed to handle either streaming or batch processing but rarely both, which forces development teams to choose between low-latency responsiveness and high-quality analytical output. That trade-off is no longer acceptable when modern applications demand both capabilities at once. This project addresses that gap directly by introducing a Lambda-style news analytics pipeline that runs a real-time stream and a batch process side by side, giving users the best of both worlds within a single unified framework.

At the core of the pipeline sits Apache Kafka, which acts as the central event streaming backbone and handles high-throughput ingestion of articles pulled continuously from multiple external APIs. The Speed Layer reads directly from Kafka through a dedicated consumer, pushes events into MongoDB, and surfaces them through REST endpoints built on FastAPI. This ensures that users can query the most recent news within milliseconds of it being published, making the system highly responsive for live monitoring and time-sensitive decision making.

Running in parallel, the Batch Layer takes a more deliberate and thorough approach to processing. Apache Flink handles incoming data using five-minute tumbling windows, applying structured cleaning, enrichment, and aggregation operations before writing the results into MinIO. Storage inside MinIO follows the Medallion Architecture, where Bronze holds raw ingested data, Silver holds cleaned and normalised records, and Gold holds fully enriched, analytics-ready datasets. Each layer builds on the one before it, ensuring that data quality improves progressively at every stage of the pipeline.

Beyond the technical architecture, the broader motivation behind this project is to make civic and news intelligence more accessible and actionable. Raw data from APIs and RSS feeds holds significant informational value, but that value remains locked unless there is a system capable of processing, organising, and surfacing it in a meaningful way. This pipeline transforms scattered, unstructured inputs into clean, categorised, and trend-aware outputs that can genuinely support decision making at scale. Whether the use case involves tracking breaking news, monitoring regional incidents, or identifying patterns over time, the system is built to serve as a reliable foundation for any application that depends on timely and accurate news intelligence.

1.2 Organization of Report

- Chapter 1: Introduction – This chapter provides a foundational description of the news analytics landscape. It establishes the context of the project, the motivation behind building a scalable pipeline, and outlines the overall organization of the report.
- Chapter 2: Literature Review – This chapter presents a survey of existing news processing systems and identifies critical research gaps regarding real-time scalability. It formally defines the Problem Statement, the Scope of the project, and the specific Objectives aimed at bridging the gap between batch and streaming analytics.
- Chapter 3: Proposed System – This chapter details the technical core of the project, including the Lambda Architecture framework and the ingestion algorithms. It provides a comprehensive System Architecture and Detailed Design using UML diagrams. Furthermore, it explains the Methodology and Procedures used to implement the Kafka-Flink-MinIO pipeline.
- Chapter 4: Results & Discussions – This chapter specifies the software and hardware environments required for the system. It showcases the Partial Implementation of the pipeline, including the FastAPI backend and Streamlit frontend, while defining the Performance Evaluation Parameters used to validate system throughput and latency.
- Chapter 5: Conclusion & Plan for Next Semester – The final chapter summarizes the contributions of the current work. It concludes with a structured Implementation Plan and Gantt Chart detailing the milestones for the upcoming semester, focusing on full-scale deployment and advanced analytics.

Chapter 2

Literature Review

Research has been conducted to provide robust frameworks for real-time news monitoring and large-scale sentiment analysis using traditional ETL and Kappa architectures. While these systems have been prototyped using standard stream processing tools, industries remain dissatisfied due to the high latency in historical data retrieval and the extreme resource complexity required to maintain dual codebases. Because of these limitations—specifically the trade-off between real-time responsiveness and deep analytical accuracy—scalable news analytics pipelines remain a critical area of ongoing research and optimization.

2.1 Survey Existing system

Treats citizens' tweets as "human sensors" to detect and grade urban events for Wireless Sensor Networks (WSNs). Uses spatio-temporal clustering and supervised classification to prioritize detected events. Demonstrated success in detecting earthquakes, traffic, and health incidents.

A fog computing architecture designed to reduce latency and energy consumption by processing IoT data closer to end devices. Demonstrated efficiency gains compared to cloud- centric models through simulations in iFogSim.

Integrates Apache Flink (stream processing) with Apache Iceberg (data management) to handle both streaming and batch workloads. Provides low-latency analytics, schema evolution, and scalable querying for smart city data

Extends OLAP techniques to real-time smart city data. Enables multidimensional operations (slice, dice, drill-down) on streaming facility utilization data. Validated with a case study in Tsukuba City, Japan.

Combines CNN-based syntactic features and CRF-based semantic features to extract and classify city events from Twitter streams. Achieved ≈81% accuracy in event detection and demonstrated potential for early detection of traffic events.

2.2 Limitation of existing system

- **Limited Data Sources:** Existing systems rely heavily on social media, leading to coverage gaps, noisy text, and missing geolocation information.
- **Lack of Real-World Development**: Many frameworks are validated primarily through simulations, which do not capture real-world latency, heterogeneity, or fault tolerance.
- **Handling Unstructured and Noisy data**: Stream processing and data lake systems often assume clean, structured input and struggle with raw, unstructured civic data.
- **Scalability Challenges**: Prior systems face difficulties managing multiple heterogeneous, high-velocity data streams in real-time.
- **Limited Adaptability**: Deep learning or specialized models require continual retraining for new event types, locations, or languages.

2.3 Problem statement and objectives

The rapid growth of online news platforms and the constant generation of articles from multiple sources pose significant challenges for traditional data processing systems. Conventional architectures often struggle to provide low-latency access to real-time events while simultaneously maintaining a structured historical record for analytics. Without a unified pipeline, it becomes difficult to monitor breaking news promptly, track trends over time, or generate reliable insights for decision-making.

Furthermore, news data is highly heterogeneous and arrives at unpredictable intervals, requiring robust mechanisms for event ingestion, cleaning, aggregation, and storage. Existing systems that focus only on real-time streaming may compromise data quality, while batch-only systems fail to deliver timely updates. There is a clear need for a pipeline that balances speed, reliability, and historical consistency, enabling both immediate visibility and comprehensive analytical capabilities.

2.3.1 Objectives

The primary objective of this project is to design and implement a scalable news analytics pipeline that supports both real-time updates and historical analytics. The system ingests news articles from multiple APIs, streams them reliably using Kafka, and processes them in parallel. It employs a Speed Layer for instant updates and a Batch Layer for long-term storage and analytics.

- Ensure low-latency ingestion and retrieval of news articles for live dashboards using MongoDB, enabling near real-time access to information.
- Implement batch processing with Apache Flink to clean, enrich, and aggregate data into structured layers (Bronze, Silver, Gold) stored in MinIO, ensuring both raw and refined datasets are maintained.
- Provide REST APIs via FastAPI for real-time and historical analytics, supporting flexible queries and integration with visualization dashboards.

- Maintain scalability, fault tolerance, and reliability across the pipeline to handle high data volumes while ensuring uninterrupted performance and data integrity.
- Integrate automated data validation and enrichment mechanisms (such as deduplication, entity recognition, and topic classification) to improve the overall quality, consistency, and usability of ingested news data for downstream analytics.

2.4 Scope

The scope of this project encompasses the design, development, and validation of an end-to-end news analytics ecosystem that bridges the gap between high-velocity data ingestion and meaningful user consumption. Specifically, as data transitions through various stages of the pipeline-moving from raw, unstructured JSON payloads from seven disparate APIs to refined, high-value datasets in the Gold layer-the solution ensures data integrity and analytical consistency while keeping the experience seamless for the end-user.
The project scope includes:

- **Multi-Source Ingestion:** Orchestrating the simultaneous capture of real-time feeds from seven independent global news providers into a centralized Apache Kafka streaming backbone.
- **Architectural Duality:** Implementing a functional Lambda Architecture that enables the system to handle immediate event updates via the Speed Layer (MongoDB) while concurrently managing massive historical processing via the Batch Layer (MinIO).
- **Data Refinement & Lineage:** Developing a Medallion Lakehouse structure within MinIO to systematically clean, normalize, and enrich news articles, ensuring that the transition from raw "Bronze" data to curated "Gold" analytics is automated and verifiable.
- **Unified Access Layer:** Constructing a robust FastAPI middleware that abstracts the complexity of dual-layer storage, providing external applications with a single, high-performance point of entry for both live and archival data.

Chapter 3

Proposed System

3.1 Proposed System Statement

The proposed system is a scalable and fault-tolerant news analytics pipeline designed to process high-velocity, unstructured news data from multiple sources. The system integrates real-time streaming and batch processing to provide both instant updates and historical insights. It leverages a Lambda Architecture-based approach, where:

- A Speed Layer enables real-time data ingestion and low-latency access.
- A Batch Layer performs large-scale processing for accurate and structured analytics.

The system ensures:

- High-throughput ingestion using Apache Kafka
- Low-latency storage using MongoDB
- Structured long-term storage using MinIO (Bronze–Silver–Gold layers)
- Unified access via FastAPI
- Visualization using Streamlit

This architecture enables efficient monitoring, trend detection, and decision-making from continuous news streams.

3.2 Framework

The proposed data processing framework is designed to handle continuous streams of news articles efficiently, ensuring both real-time responsiveness and comprehensive historical analytics. By adopting a Lambda-like architecture with separate speed and batch layers, the system achieves low-latency insights for live dashboards while maintaining structured, analytics-ready datasets for long-term reporting and decision-making

1. Data Ingestion Layer

- Collects news articles from multiple APIs.
- Standardizes and serializes data for consistency.
- Publishes events to a Kafka topic (*news-events*), decoupling producers from downstream consumers.

2. Speed Layer (Real-Time Processing)

- A lightweight Kafka consumer subscribes to *news-events*.
- Writes parsed articles into MongoDB, optimized for rapid reads/writes.
- Exposed via REST APIs to power live dashboards and immediate insights.

3. Batch Layer (Historical Processing)

- Apache Flink consumes events from Kafka in time windows.
- Implements Medallion Architecture in MinIO:
- Bronze: Raw, immutable ingested events.
- Silver: Cleaned, normalized, and structured datasets.
- Gold: Aggregated, analytics-ready datasets (e.g., trends, summaries).

4. Serving Layer

- Real-time queries served from MongoDB.
- Historical and analytical queries served from MinIO Gold datasets.
- Unified API access (FastAPI) ensures seamless query routing.

Component	**Technology**
Data Source	World News API (REST , JSON)
Message Broker	Apache Kafka
Real-Time Consumer	Kafka Python
Batch Processor	Apache Flink (PyFlink)
Primary Database	MongoDB
Data Lakehouse	Minio Object Storage (Bronze / Silver / Gold)
API Framework	FastAPI
User Interface	Streamlit
Containerization	Docker / Docker Compose

Table 1: Technology Stack Of The Civic Intelligence Framework

3.3 Design Details

3.3.1 System Architecture

The architecture processes data in two parallel streams: a Speed Layer for real-time insights and a Batch Layer for historical analytics. Data is ingested from sources via APIs and sent to Apache Kafka, which buffers, distributes, and stores the data reliably. The Speed Layer uses a real-time consumer to process incoming messages and stores them in MongoDB for fast queries and dashboard updates. The Batch Layer uses Apache Flink to process micro-batches, performing aggregations, enrichments, and transformations, storing results in a Data Lakehouse following the Medallion Architecture (Bronze → Silver → Gold). Finally, an API unifies data from both layers, powering a dashboard that shows live and historical trends

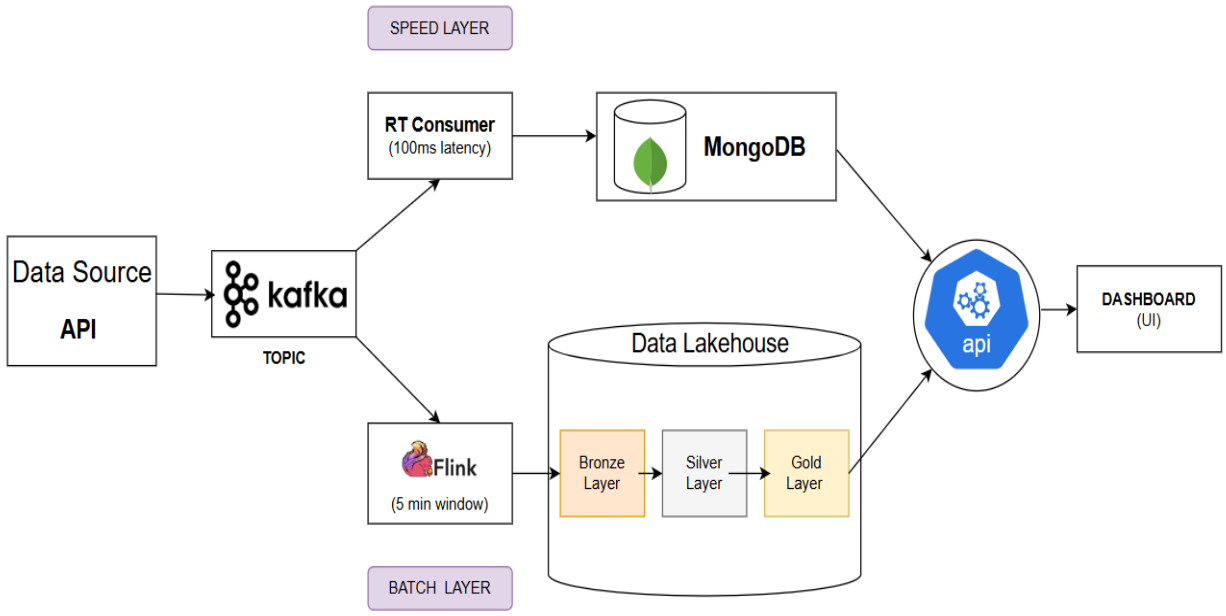


Figure. 3.3.1 System Architecture

3.3.2 Detailed Design

Here's a structured overview of the architecture:

Overview of the Architecture
Designed with two parallel streams:

- Speed Layer: Provides real-time results.
- Batch Layer: Provides historical, accurate, and enriched analytics.

1. Data Ingestion
    - Data Source (API): Streams events continuously (user logs, IoT metrics, transactions, social media feeds).
    - Apache Kafka (Topic):

- Buffer: Handles high volumes and bursts of traffic.
- Distributor: Allows multiple downstream consumers to read independently.
- Durability: Stores data reliably to prevent loss if downstream fails.

2. Dual Processing Layers
   - Data from Kafka is processed in parallel by two layers: Speed and Batch.

3. Speed Layer (Top Path)
   - Purpose: Provides immediate insights
   - RT Consumer: Lightweight app, reads Kafka messages instantly (~100ms latency), performs minimal processing.
   - MongoDB:
   - High-speed writes for real-time ingestion.
   - Flexible schema for raw, changing data.
   - Fast queries for dashboards and applications.

4. Batch Layer (Bottom Path)
   - Purpose: Provides comprehensive analytics.
   - Apache Flink: Reads from Kafka, processes in micro-batches, performs:
   - Aggregations: Sums, averages, counts.
   - Enrichment: Joins with other datasets for context.
   - Transformations: Cleans, filters, restructures data.
   - Data Lakehouse & Medallion Architecture:
   - Bronze: Raw, unfiltered data (archival & reprocessing).
   - Silver: Cleaned, validated, structured data (source of truth).
   - Gold: Refined, aggregated data ready for BI/analytics.

5. Serving & Presentation Layer
   - API: Single access point for front-end, fetches:
   - Latest data from MongoDB (Speed Layer).
   - Historical, aggregated data from Gold Layer (Batch Layer).
   - Dashboard: Displays combined real-time and historical trends, e.g., live events alongside weekly charts.

3. 4 Methodology

To build a robust system for both real-time news delivery and historical analytics, the following methodology is followed. The architecture ensures seamless ingestion, processing, storage, and serving of news data while maintaining Low latency and High reliability.

Methodology for Real time and Batch News Data Pipeline

1. Data Collection

- Identify and connect to multiple news APIs or sources for continuous ingestion.
- Implement scripts to fetch news articles periodically or in real-time.
- Handle API authentication, rate-limiting, and error management.

2. Data Streaming with Kafka

- Publish all fetched articles to an Apache Kafka topic.
- Kafka acts as a durable message bus, buffering data and decoupling producers
- Configure partitions and retention policies for high throughput and reliability.

3. Real-Time Processing (Speed Layer)

- Deploy a lightweight Kafka consumer to read incoming messages instantly.
- Parse and minimally process articles for immediate usability.
- Store processed data in MongoDB for fast queries and dashboard updates.

4. Historical Processing (Batch Layer)

- Use Apache Flink to process Kafka streams in micro-batches or streaming mode.
- Perform cleaning, enrichment, aggregation, and transformation of raw data.
- Store results in MinIO Data Lakehouse using Medallion Architecture:
- Bronze: Raw, immutable data.
- Silver: Cleaned and structured data.
- Gold: Aggregated, analytics-ready data for insights.

5. Unified Serving Layer

- Deploy FastAPI as a single access point for real-time and historical data.
- Route requests intelligently:
- Latest news → MongoDB
- Historical trends → Gold layer in MinIO
- Provide endpoints for filtering, search, and aggregation for dashboards or client apps.

6. Visualization and Monitoring

- Connect APIs to a React dashboard or other front-end clients.
- Monitor system performance, latency, and errors.
- Implement logging, alerting, and automated retries for reliability.

7. Iterative Improvement

- Regularly evaluate data quality, schema flexibility, and scalability.
- Optimize Flink jobs, Kafka configurations, and MongoDB indexes.
- Introduce additional enrichments like sentiment analysis or topic modeling as needed.

describe every procedure or algorithm with necessary diagram. also explain different technology or languages you used for implementation/Deployment.

3.4.1 Procedures

1. **Data Collection**
   - News articles are collected from the API and sent to Kafka for processing.

2. **Real-Time Processing (Speed Layer)**
   - Data is consumed instantly from Kafka.
   - Basic processing is done and stored in MongoDB for live updates.

3. **Batch Processing (Batch Layer)**
   - Data is processed using Flink.
   - Cleaned and stored in Data Lakehouse (Bronze → Silver → Gold).

4. **Data Structuring**
   - Each event is organized using a fixed schema:
   - ID, type, title, content, time (UTC), location, severity, source link.

5. **Event Classification**
   - Keywords are counted for each category.
   - Event is assigned to the category with highest keyword frequency.

   - **Severity Calculation**

   - Severity of each event is calculated using:

$$S(e) = \alpha \cdot f_{\{kw\}(e)} + (1 - \alpha) \cdot g_{\{loc\}(e)}$$

   Where:
   - $(f_{\{kw\}(e)})$= keyword importance
   - $(g_{\{loc\}(e)})$= location importance
   - (α= 0.6)

6. **Window Processing**

   - Data is divided into fixed time windows:

$$W_{\{n\}} = [n \cdot T, (n+1) \cdot T), \qquad T = 300$$

7. **Duplicate Removal**
   - A hash is generated using article URL and title.
   - Duplicate records in current and previous window are removed.

8. **Knowledge Fusion**
   - Reliability of sources is applied using:

$$H(x) = \Sigma \cdot q_i \cdot h_i(x)$$

Where:

- (q_i)= source reliability
- $(h_{i(x)})$= event presence (0 or 1)

9. **API Integration**
   - FastAPI combines real-time and batch data.
   - Provides a single endpoint for frontend use.

10. **Dashboard Display**
    - Shows:
    - Live events (from speed layer)
    - Historical data (from batch layer)

3.4.2 Psuedocode

1. System Initialization

The system boots every component in a fixed sequence so nothing starts before its dependency is ready. Kafka must be live before consumers connect, and the API only opens once storage is confirmed.

```
START SYSTEM

INITIALIZE Data Sources
INITIALIZE Kafka Cluster
INITIALIZE Speed Layer Consumer
INITIALIZE Batch Layer (Flink Jobs)
INITIALIZE Data Lakehouse
INITIALIZE Databases (MongoDB)
INITIALIZE API Server

SYSTEM STATE = RUNNING
```

2. Data Ingestion Agent

This agent runs non-stop, pulling civic data from three source types and converting each response into one common format before anything downstream sees it. Adding a new source later requires no changes to any other part of the system.

```
AGENT: DataIngestionAgent

WHILE SYSTEM STATE == RUNNING:
  FOR each source IN [News_API, RSS_Feed, Social_Alerts]:

    raw_event = FETCH(source)

    IF raw_event IS NULL:
      CONTINUE

    standardized_event = NORMALIZE(raw_event)
    // Fields: raw_text, source_name, published_time, optional_location

    PRODUCE Kafka_Topic("civic-events", standardized_event)
```

```
  WAIT polling_interval
```

3. Kafka Event Stream

Kafka sits in the middle of the entire pipeline and lets two separate consumers — the Speed Layer and the Batch Layer — read the same stream independently without blocking each other. If either consumer crashes, it resumes from the exact point it stopped.

```
KAFKA TOPIC: civic-events

PROPERTIES:
  - Ordered
  - Fault-tolerant
  - Horizontally scalable
  - Supports multiple independent consumers

CONSUMERS:
  - Speed Layer Consumer   (real-time path)
  - Flink Batch Processor  (batch path)
```

4. Speed Layer Consumer

This layer is built purely for speed — validate, resolve location if missing, write to MongoDB, done. Heavy processing is deliberately left to the Batch Layer so the live dashboard gets updates within seconds.

```
AGENT: SpeedLayerConsumer

SUBSCRIBE TO Kafka_Topic("civic-events")

FOR each event RECEIVED:

  IF NOT VALID(event):
  DISCARD event

  enriched_event = RESOLVE_LOCATION(event)

  INSERT INTO MongoDB.Collection("live_events") enriched_event
```

Location Resolution

Many news articles carry no GPS tag, so this function reads the raw text itself using Named Entity Recognition to spot place names and a Gazetteer to convert them into coordinates.

```
FUNCTION RESOLVE_LOCATION(event):

  IF event.location IS NOT NULL:
   RETURN event

  ELSE:
  inferred_location = NLP_GEO_RESOLVER(event.raw_text)
  event.location = inferred_location
  RETURN event
```

5. Flink Batch Processor

Flink collects five minutes of events into one window so it can detect and remove duplicate reports — something impossible when processing one event at a time. Every deduplicated event then goes through feature extraction before being written to Bronze.

```
FLINK JOB: CivicBatchProcessor
WINDOW TYPE: Tumbling Window  |  WINDOW SIZE: 5 minutes

READ STREAM FROM Kafka_Topic("civic-events")

FOR EACH time_window (5 minutes):

  events = COLLECT_EVENTS(time_window)
  deduplicated_events = DEDUPLICATE(events)

  structured_events = [
  FOR each event IN deduplicated_events:
    structured_event = EXTRACT_FEATURES(event)
    structured_events.APPEND(structured_event)

  WRITE TO Lakehouse.Layer("BRONZE") structured_events
]
```

6. Feature Extraction Function

Unstructured text becomes a typed, queryable record with six defined fields. The original raw text is always kept alongside the extracted fields so the pipeline can be re-run if any model is updated later.

```
FUNCTION EXTRACT_FEATURES(event):

  RETURN {
    timestamp  : EXTRACT_TIMESTAMP(event),
    location   : EXTRACT_LOCATION(event),
    event_type : CLASSIFY_EVENT(event.raw_text),
    severity   : ESTIMATE_SEVERITY(event),
    // Low | Medium | High
    raw_text   : event.raw_text,
    source     : event.source
```

7. Data Lakehouse - Bronze, Silver, Gold

Data only ever moves forward through three layers, each adding one level of refinement. Bronze stores everything raw, Silver cleans it, and Gold enriches it so the API can serve results without any further computation.

7a. Bronze Layer

Every Flink output lands here untouched. It serves as the permanent archive — if Silver or Gold ever needs to be rebuilt, it all starts from Bronze without re-fetching external sources.

```
PURPOSE: Backup / Audit / Replay

WRITE TO Lakehouse.Layer("BRONZE") structured_events
```

7b. Silver Layer - Cleaning Agent

The Cleaning Agent applies three checks to every Bronze record: drop nulls, convert timestamps to UTC, and confirm coordinates are within valid geographic bounds. Anything that fails gets dropped here so bad data never reaches Gold.

```
AGENT: CleaningAgent

READ FROM Lakehouse.Layer("BRONZE")

FOR each record:
  cleaned_record = CLEAN(record)

  WRITE TO Lakehouse.Layer("SILVER") cleaned_record
```

7c. Gold Layer - Enrichment Agent

The Enrichment Agent groups events by city, normalises severity scores across the full batch, and tags any emerging spikes as trending. This is the only layer the API reads for historical queries.

```
AGENT: EnrichmentAgent

READ FROM Lakehouse.Layer("SILVER")

FOR each record:
  enriched_record = ENRICH(record)

  WRITE TO Lakehouse.Layer("GOLD") enriched_record
```

8. Database Storage

Splitting the two means each database is optimised for exactly the workload it sees.

```
DATABASE: MongoDB
  Collections: live_events | recent_events
  Used for: Live dashboard, geospatial queries

DATABASE: MinIO
  Used for: Historical analytics, joins, reports
  Example: Power outages in Delhi — last 30 days by severity
```

9. API Endpoints

Three endpoints cover every consumer need. The API does no processing of its own — it reads from the correct storage tier and returns the result, keeping response times predictable.

```
GET /live-events
  → Reads from MongoDB | Lowest latency

GET /historical-events
  → Reads from Gold Layer | Deduplicated and enriched

GET /filters
  → Parameters: event_type | city | severity | time_window
```

Chapter 4

Results & Discussions

4.1 Software and Hardware Set up

4.1.1 Software Requirements

Backend:

- Python `
- Apache Kafka
- Apache Zookeeper
- Apache Flink
- MongoDB
- MinIO
- FastAPI

Libraray Tools:

- Kafka-python, pymongo, minio, pandas
- Docker Desktop & Docker Compose

Frontend:

- Streamlit

4.1.2 Hardware Requirements

- Processor: Quad-core CPU, 8 GB RAM, 100 GB SSD, 1 Gbps Ethernet RAM: 8 GB

- Storage: 100 GB SSD for fast data ingestion and storage

4.2 Results

The figure shows that all Docker containers for your news-realtime-pipeline project were successfully built and started using Docker Compose, indicating the entire pipeline is up and running.

```
#40 [api] resolving provenance for metadata file
#40 DONE 0.2s
[+] Building 5/5
 ✓ news-realtime-pipeline-speed-consumer  Built                                                    0.0s
 ✓ news-realtime-pipeline-api             Built                                                    0.0s
 ✓ news-realtime-pipeline-batch-layer     Built                                                    0.0s
 ✓ news-realtime-pipeline-speed-producer  Built                                                    0.0s
 ✓ news-realtime-pipeline-dashboard       Built                                                    0.0s
PS C:\Users\Disha\Desktop\review\news-realtime-pipeline> docker-compose up -d
time="2025-10-04T14:47:21+05:30" level=warning msg="C:\\Users\\Disha\\Desktop\\review\\news-realtime-pipeline\\docker-compose.yml: the attribute `version` is obsolete
, it will be ignored, please remove it to avoid potential confusion"
[+] Running 11/11
 ✓ Network news-pipeline     Created                                                               0.4s
 ✓ Container mongodb         Healthy                                                              30.5s
 ✓ Container zookeeper       Healthy                                                              29.0s
 ✓ Container minio           Healthy                                                              30.5s
 ✓ Container api             Healthy                                                              25.9s
 ✓ Container kafka           Healthy                                                              85.1s
 ✓ Container dashboard       Started                                                              25.7s
 ✓ Container batch-layer     Started                                                              86.0s
 ✓ Container kafka-ui        Started                                                              86.3s
 ✓ Container speed-consumer  Started                                                              86.0s
 ✓ Container speed-producer  Started                                                              86.0s
```

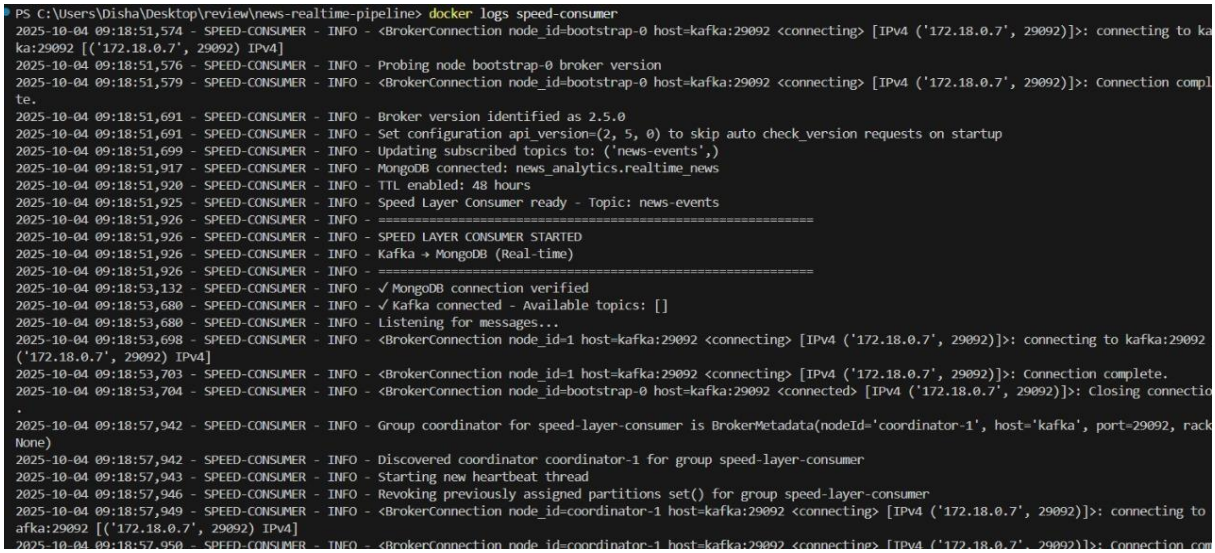
Figure 4.1: Docker Setup

The figure shows the Docker logs for the speed-producer service in your news-realtime-pipeline. It indicates that the producer is successfully fetching news data from multiple endpoints.

```
PS C:\Users\Disha\Desktop\review\news-realtime-pipeline> docker-compose logs -f speed-producer
speed-producer  | ============================================================
speed-producer  | 2025-10-04 09:18:58,034 - SPEED-PRODUCER - INFO - CYCLE 1 - Fetching from all 7 endpoints
speed-producer  | ============================================================
speed-producer  | 2025-10-04 09:18:58,034 - SPEED-PRODUCER - INFO - Fetching search_news: Cricket India
speed-producer  | 2025-10-04 09:18:59,797 - SPEED-PRODUCER - INFO -   → Found 20 articles
speed-producer  | 2025-10-04 09:18:59,827 - SPEED-PRODUCER - INFO - ✓ Sent search_news -> Kafka | Total: 1
speed-producer  | 2025-10-04 09:19:01,807 - SPEED-PRODUCER - INFO - Fetching feed_rss: https://www.news18.com
speed-producer  | 2025-10-04 09:19:02,398 - SPEED-PRODUCER - INFO -   → RSS content: 185784 chars
speed-producer  | 2025-10-04 09:19:02,425 - SPEED-PRODUCER - INFO - ✓ Sent feed_rss -> Kafka | Total: 2
speed-producer  | 2025-10-04 09:19:04,426 - SPEED-PRODUCER - INFO - Fetching search_news_sources: News18
speed-producer  | 2025-10-04 09:19:04,701 - SPEED-PRODUCER - WARNING -   → search_news_sources returned no data
speed-producer  | 2025-10-04 09:19:06,702 - SPEED-PRODUCER - INFO - Fetching geo_coordinates: Bangalore, India
speed-producer  | 2025-10-04 09:19:07,106 - SPEED-PRODUCER - INFO -   → Coordinates: 12.9628669, 77.57750899999999
speed-producer  | 2025-10-04 09:19:07,126 - SPEED-PRODUCER - INFO - ✓ Sent geo_coordinates -> Kafka | Total: 3
speed-producer  | 2025-10-04 09:19:09,126 - SPEED-PRODUCER - INFO - Fetching top_news
speed-producer  | 2025-10-04 09:19:09,601 - SPEED-PRODUCER - INFO -   → Found 28 news clusters
speed-producer  | 2025-10-04 09:19:09,722 - SPEED-PRODUCER - INFO - ✓ Sent top_news -> Kafka | Total: 4
speed-producer  | 2025-10-04 09:19:11,723 - SPEED-PRODUCER - INFO - Fetching extract_news: https://www.news18.com/agency-feeds/rampant-india-...
speed-producer  | 2025-10-04 09:19:12,284 - SPEED-PRODUCER - INFO -   → Extracted article: Rampant India crush West Indies to win f...
speed-producer  | 2025-10-04 09:19:12,301 - SPEED-PRODUCER - INFO - ✓ Sent extract_news -> Kafka | Total: 5
speed-producer  | 2025-10-04 09:19:14,301 - SPEED-PRODUCER - INFO - Fetching extract_news_links
speed-producer  | 2025-10-04 09:19:14,801 - SPEED-PRODUCER - INFO -   → Extracted 3 URLs
speed-producer  | 2025-10-04 09:19:14,817 - SPEED-PRODUCER - INFO - ✓ Sent extract_news_links -> Kafka | Total: 6
speed-producer  | 2025-10-04 09:19:16,818 - SPEED-PRODUCER - INFO -
speed-producer  | ============================================================
speed-producer  | 2025-10-04 09:19:16,818 - SPEED-PRODUCER - INFO - CYCLE 1 COMPLETE
speed-producer  | 2025-10-04 09:19:16,818 - SPEED-PRODUCER - INFO - Total Sent: 6 | Errors: 0
speed-producer  | 2025-10-04 09:19:16,818 - SPEED-PRODUCER - INFO - Next cycle in 180s
speed-producer  | 2025-10-04 09:19:16,818 - SPEED-PRODUCER - INFO - ============================================================
speed-producer  |
```

Figure 4.2: Speed Producer Logs

This figure shows the successful startup of the Speed Layer Consumer, which connects Kafka (news- events topic) to MongoDB real-time data transfer. It verifies broker and database connections, joins the consumer group, and begins listening for live messages



```
PS C:\Users\Disha\Desktop\review\news-realtime-pipeline> docker logs speed-consumer
2025-10-04 09:18:51,574 - SPEED-CONSUMER - INFO - <BrokerConnection node_id=bootstrap-0 host=kafka:29092 <connecting> [IPv4 ('172.18.0.7', 29092)]>: connecting to ka
ka:29092 [('172.18.0.7', 29092) IPv4]
2025-10-04 09:18:51,576 - SPEED-CONSUMER - INFO - Probing node bootstrap-0 broker version
2025-10-04 09:18:51,579 - SPEED-CONSUMER - INFO - <BrokerConnection node_id=bootstrap-0 host=kafka:29092 <connecting> [IPv4 ('172.18.0.7', 29092)]>: Connection compl
te.
2025-10-04 09:18:51,691 - SPEED-CONSUMER - INFO - Broker version identified as 2.5.0
2025-10-04 09:18:51,691 - SPEED-CONSUMER - INFO - Set configuration api_version=(2, 5, 0) to skip auto check_version requests on startup
2025-10-04 09:18:51,699 - SPEED-CONSUMER - INFO - Updating subscribed topics to: ('news-events',)
2025-10-04 09:18:51,917 - SPEED-CONSUMER - INFO - MongoDB connected: news_analytics.realtime_news
2025-10-04 09:18:51,920 - SPEED-CONSUMER - INFO - TTL enabled: 48 hours
2025-10-04 09:18:51,925 - SPEED-CONSUMER - INFO - Speed Layer Consumer ready - Topic: news-events
2025-10-04 09:18:51,926 - SPEED-CONSUMER - INFO - ============================================================
2025-10-04 09:18:51,926 - SPEED-CONSUMER - INFO - SPEED LAYER CONSUMER STARTED
2025-10-04 09:18:51,926 - SPEED-CONSUMER - INFO - Kafka → MongoDB (Real-time)
2025-10-04 09:18:51,926 - SPEED-CONSUMER - INFO - ============================================================
2025-10-04 09:18:53,132 - SPEED-CONSUMER - INFO - ✓ MongoDB connection verified
2025-10-04 09:18:53,680 - SPEED-CONSUMER - INFO - ✓ Kafka connected - Available topics: []
2025-10-04 09:18:53,680 - SPEED-CONSUMER - INFO - Listening for messages...
2025-10-04 09:18:53,698 - SPEED-CONSUMER - INFO - <BrokerConnection node_id=1 host=kafka:29092 <connecting> [IPv4 ('172.18.0.7', 29092)]>: connecting to kafka:29092
('172.18.0.7', 29092) IPv4]
2025-10-04 09:18:53,703 - SPEED-CONSUMER - INFO - <BrokerConnection node_id=1 host=kafka:29092 <connecting> [IPv4 ('172.18.0.7', 29092)]>: Connection complete.
2025-10-04 09:18:53,704 - SPEED-CONSUMER - INFO - <BrokerConnection node_id=bootstrap-0 host=kafka:29092 <connected> [IPv4 ('172.18.0.7', 29092)]>: Closing connectio
.
2025-10-04 09:18:57,942 - SPEED-CONSUMER - INFO - Group coordinator for speed-layer-consumer is BrokerMetadata(nodeId='coordinator-1', host='kafka', port=29092, rack
None)
2025-10-04 09:18:57,942 - SPEED-CONSUMER - INFO - Discovered coordinator coordinator-1 for group speed-layer-consumer
2025-10-04 09:18:57,943 - SPEED-CONSUMER - INFO - Starting new heartbeat thread
2025-10-04 09:18:57,946 - SPEED-CONSUMER - INFO - Revoking previously assigned partitions set() for group speed-layer-consumer
2025-10-04 09:18:57,949 - SPEED-CONSUMER - INFO - <BrokerConnection node_id=coordinator-1 host=kafka:29092 <connecting> [IPv4 ('172.18.0.7', 29092)]>: connecting to
afka:29092 [('172.18.0.7', 29092) IPv4]
2025-10-04 09:18:57,950 - SPEED-CONSUMER - INFO - <BrokerConnection node_id=coordinator-1 host=kafka:29092 <connecting> [IPv4 ('172.18.0.7', 29092)]>: Connection com
```

Figure 4.3: Speed Layer Consumer Logs

This figure shows the Kafka topic news-events, which stores real-time news messages produced by the ingestion layer. Each record includes a timestamp, partition, and offset, representing messages ready to be consumed by the Speed Layer Consumer for processing and storage in MongoDB.

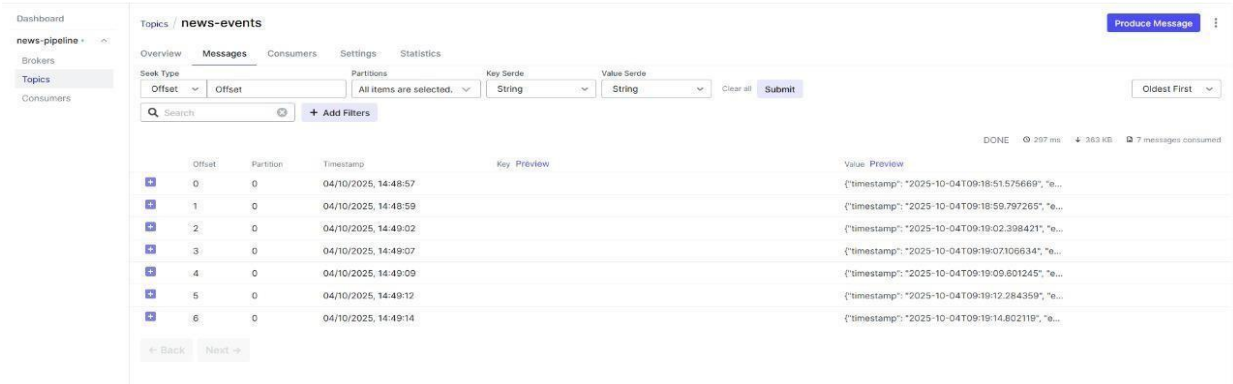


Figure 4.4: Kafka Topic — news-events

This figure shows the detailed JSON message stored in the Kafka topic news-events. It includes metadata such as timestamp, source (world_news_api), and layer (speed), along with structured news data containing fields like id, title, and text. This message represents a real-time news article ready for processing and storage in MongoDB.

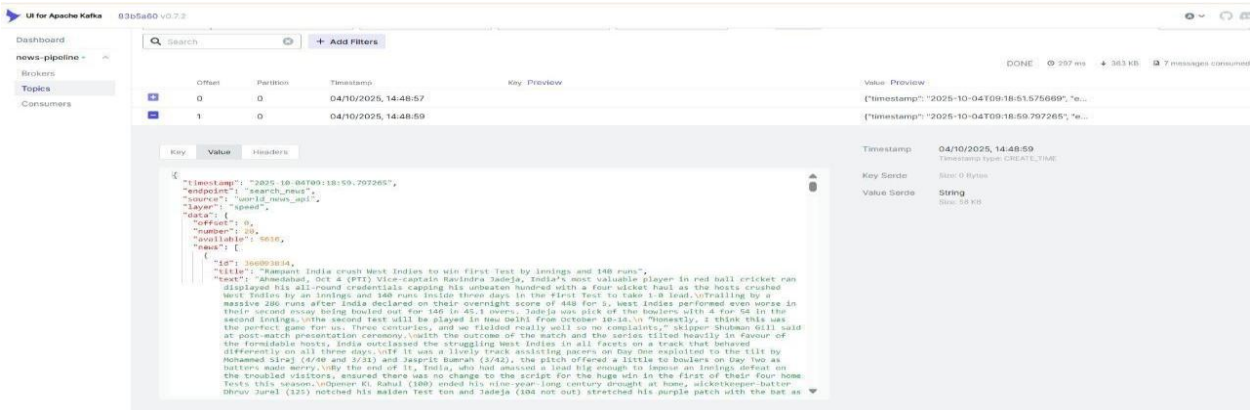


Figure 4.5: Kafka Message Data (news-events Topic)

The figure shows the buckets in MinIO Object Store, which stores data from multiple news endpoints. It acts as the storage storage layer.

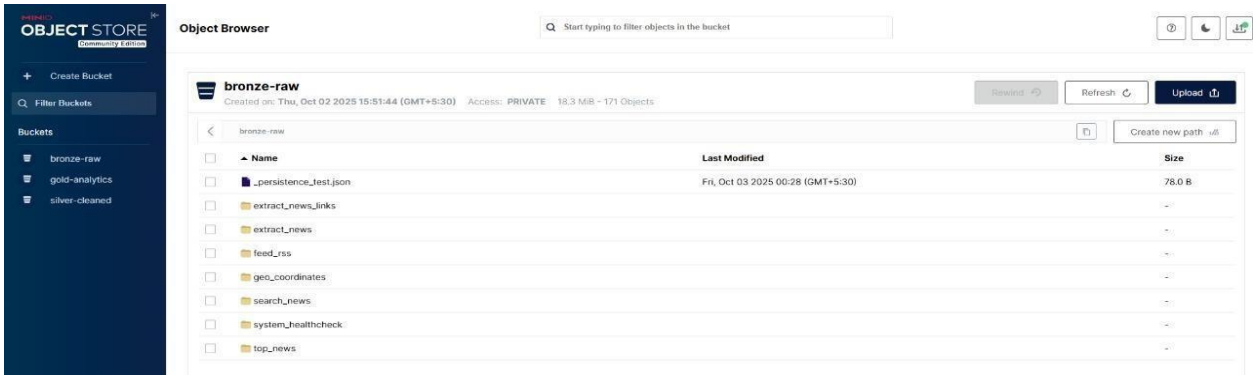


Figure 4.6: MinIO Object Store

The figure shows the execution of the verify_minio_layers.py script, which validates datasets across the bronze, silver, and gold layers. All data categories such as extract_news_links, feed_rss, and top_news are marked as VALID, confirming successful data flow and integrity in the pipeline.

```
PS C:\Users\Disha\Desktop\review\news-realtime-pipeline\scripts> python verify_minio_layers.py

extract_news_links:
   Bronze (Raw Data): ✓ VALID
   Silver (Cleaned Data): ✓ VALID
   Gold (Analytics): ✓ VALID

feed_rss:
   Bronze (Raw Data): ✓ VALID
   Silver (Cleaned Data): ✓ VALID
   Gold (Analytics): ✓ VALID

geo_coordinates:
   Bronze (Raw Data): ✓ VALID
   Silver (Cleaned Data): ✓ VALID
   Gold (Analytics): ✓ VALID

search_news:
   Bronze (Raw Data): ✓ VALID
   Silver (Cleaned Data): ✓ VALID
   Gold (Analytics): ✓ VALID

system_healthcheck:
   Bronze (Raw Data): ✓ VALID
   Silver (Cleaned Data): ✓ VALID
   Gold (Analytics): ✓ VALID

top_news:
   Bronze (Raw Data): ✓ VALID
   Silver (Cleaned Data): ✓ VALID
   Gold (Analytics): ✓ VALID

================================================================
```

Figure 4.7: Validation of MinIO Data Layers

The figure displays a MongoDB query executed via Docker to fetch the latest record from the news_analytics database.It shows detailed metadata of a news feed document (feed_rss) including the source URL, timestamp, and content information, confirming successful data ingestion and storage in MongoDB.

```
PS C:\Users\Disha\Desktop\review\news-realtime-pipeline> docker exec mongodb mongosh -u admin -p password123 --quiet --eval "db.getSiblingDB('news_analytics').realtim
e_news.countDocuments()"
203
PS C:\Users\Disha\Desktop\review\news-realtime-pipeline> docker exec mongodb mongosh -u admin -p password123 --quiet --eval "db.getSiblingDB('news_analytics').realtim
e_news.find().sort({created_at:-1}).limit(1).pretty()"
[
  {
    _id: ObjectId('68e0e75a9a60c658f41cc9d8'),
    endpoint: 'feed_rss',
    timestamp: '2025-10-04T09:22:34.825244',
    created_at: ISODate('2025-10-04T09:22:34.941Z'),
    source: 'world_news_api',
    layer: 'speed',
    data: {
      source_url: 'https://www.news18.com',
      content_length: 186886,
      preview: '<rss version="2.0" xmlns:dc="http://purl.org/dc/elements/1.1/" xmlns:media="http://search.yahoo.com/mrss/" xmlns:atom="http://www.w3.org/2005/Atom"><c
hannel><title><![CDATA[News: Get Latest News Today, Breaking News, Top News Headlines, India and World News | News18]]></title><link>https://www.news18.com</link><las
tBuildDate>Sat, 04 Oct 2025 09:22:34 +0000</lastBuildDate><pubDate>Sat, 04 Oct 2025 09:22:34 +0000</pubDate><item><title><![CDATA[‘Lewis Hamilton Vs Max Verstappen’:
Two Auto Rickshaws'
    },
    metadata: {
      endpoint: 'feed_rss',
      source_url: 'https://www.news18.com',
      content_length: 186886
    }
  }
]
PS C:\Users\Disha\Desktop\review\news-realtime-pipeline>
```

Figure 4.8: Latest news document fetched from Mongo

This figure shows API json response for analytics including speed layer and batch layer with its storage


```
Pretty print ☑

{
  "speed_layer": {
    "total_documents": 215,
    "storage": "mongodb",
    "retention": "48 hours"
  },
  "batch_layer": {
    "bronze_objects": 188,
    "silver_objects": 186,
    "gold_objects": 186,
    "storage": "minio",
    "retention": "permanent"
  }
}
```


Figure 4.9: API JSON response display for analytics

This figure shows a browser displaying a JSON response from a local API that fetches the latest news articles. It contains metadata and news details like title, summary, author, and image URL for two articles.


```
localhost:8000/api/realtime/by-endpoint/search_news?limit=2
All Bookmarks
Pretty print ☑
{
  "endpoint": "search_news",
  "count": 2,
  "data": [
    {
      "endpoint": "search_news",
      "timestamp": "2025-10-04T09:29:06.735305",
      "created_at": "2025-10-04T09:29:06.788000",
      "source": "world_news_api",
      "layer": "speed",
      "data": {
        "offset": 0,
        "number": 20,
        "available": 5620,
        "news": [
          {
            "id": 36809234,
            "title": "Explained! Why Rohit Sharma Was Removed As ODI Captain Ahead Of Australia Series",
            "text": "Rohit Sharma's era as Indian captain came to an end as the BCCI named Shubman Gill the new captain of the ODI team. Gill also replaced Rohit Sharma as Test captain post his retirement from Tests ahead of the England tour. Rohit, who led India
to the final of the ODI World Cup 2023 and won the ICC Champions Trophy 2025, is eyeing a berth in the 2027 ODI World Cup squad, along with Virat Kohli, but having retired from Tests, the veterans won't be playing a lot of cricket, making their selection in World
Cup squad doubtful. Both Kohli and Rohit will have to earn the spot by delivering good performances in the ODI series preceding the World Cup. Why Rohit Sharma Was Removed As ODI Captain?With Rohit Sharma's place in the ODI World Cup squad not guaranteed, it was in
the best interest of Indian cricket to name a new captain and give him enough time to get used to the new role. Had India continued with Rohit, and he failed to make the squad, it would have left India with an inexperienced captain in the World Cup, and it would
have been a big negative for the Men in Blue. Rohit Sharma's Record As ODI CaptainRohit Sharma took over the leadership reins of the ODI team from Virat Kohli ahead of the South Africa tour in 2021. He led India in 56 matches, winning 42 games and losing 12
matches. One match ended as a tie, while one couldn't produce a result. Rohit might have retired from ODIs had India won the World Cup in 2023, but the heartbreaking loss in the final kept the fire of winning a World Cup burning, prolonging his stay in ODIs.
India's ODI squad: Shubman Gill (Captain), Rohit Sharma, Virat Kohli, Shreyas Iyer (VC), Axar Patel, KL Rahul (WK), Nitish Kumar Reddy, Washington Sundar, Kuldeep Yadav, Harshit Rana, Mohammed Siraj, Arshdeep Singh, Prasidh Krishna, Dhruv Jurel (WK), Yashasvi
Jaiswal. India's T20I squad: Suryakumar Yadav (C), Abhishek Sharma, Shubman Gill (VC), Tilak Varma, Nitish Kumar Reddy, Shivam Dube, Axar Patel, Jitesh Sharma (WK), Varun Chakravarthy, Jasprit Bumrah, Arshdeep Singh, Kuldeep Yadav, Harshit Rana, Sanju Samson (WK),
Rinku Singh, Washington Sundar.",
            "summary": "Rohit Sharma was removed as India ODI captain ahead of the three-match ODI series against Australia as the management handed the duties to Shubman Gill, who also replaced Rohit as Test captain after his retirement.",
            "url": "https://www.timesnownews.com/sports/cricket/explained-why-rohit-sharma-was-removed-as-odi-captain-ahead-of-australia-odi-series-article-152939450",
            "image": "https://images.timesnownews.com/thumb/msid-152939558,thumbsize-229276,width-1280,height-720,resizemode-75/152939558.jpg",
            "video": null,
            "publish_date": "2025-10-04 09:19:53",
            "author": "Rahaw Uddin",
            "authors": [
              "Rahaw Uddin"
            ],
            "language": "en",
            "source_country": "in",
            "sentiment": -0.124
          },
          {
            "id": 36809192,
            "title": "Ravindra Jadeja All-Round Heroics Guide India To Commanding Win Over West Indies In First Test",
            "text": "Vice-captain Ravindra Jadeja, India's most valuable player in red ball cricket ran displayed his all-round credentials capping his unbeaten hundred with a four wicket haul as the hosts crushed West Indies by an innings and 140 runs inside three
days in the first Test to take 1-0 lead. Trailing by a massive 286 runs after India declared on their overnight score of 448 for 5, West Indies performed even worse in their second essay being bowled out for 146 in 45.1 overs. Jadeja was pick of the bowlers with 4
for 54 in the second innings. The second Test will be played in New Delhi from October 10-14. \"Honestly, I think this was the perfect game for us. Three centuries, and we fielded really well so no complaints,\" skipper Shubman Gill said at post-match presentation
ceremony. With the outcome of the match and the series tilted heavily in favour of the formidable hosts, India outclassed the struggling West Indies in all facets on a track that behaved differently on all three days. If it was a lively track assisting pacers on
Day One exploited to the tilt by Mohammed Siraj (4/40 and 3/31) and Jasprit Bumrah (3/42), the pitch offered a little to bowlers on Day Two as batters made merry. By the end of it, India, who had amassed a lead big enough to impose an innings defeat on the troubled
visitors, ensured there was no change to the script for the huge win in the first of their four home Tests this season. Opener KL Rahul (100) ended his nine-year-long century drought at home, wicketkeeper-batter Dhruv Jurel (125) notched his maiden Test ton and
Jadeja (104 not out) stretched his purple patch with the bat as India flexed batting muscle, condemning the West Indies for the hard grind to take wickets. On Day Three, hoping to extract some assistance early in the morning India declared at their overnight total
but it was the Jadeja-led Indian spinners who exploited the grip off the surface and rough patches to run riot. While Nitish Reddy's athleticism handed Siraj the first wicket among pacers, it was Jadeja (4/54) who followed up his unbeaten ton with a three-wicket
burst to initiate a swift end to West Indies' plight, reducing them to 66/5 at lunch. In fact, Jadeja, with his century and a four-for in the second innings, narrowly missed the feat of hitting a ton and taking a five-for in a Test, which he has done twice in the
```


Figure 4.10: API JSON response display for News endpoint(/search_news)

This figure shows stats from the /stats endpoint, including total documents, recent activity, and usage per API route. Key endpoints listed are feed_rss, search_news, and extract_news_links.

Figure 4.11: API JSON response display for News endpoint(/stats)

This dashboard is designed to provide real-time insights into civic analytics. It shows the most important metrics (i.e., total number of articles, sentiment trend lines, geographic activity) and summarizes system updates, as well as the number of articles in each legislative category, allowing for an overall understanding of the data and the use of the platform.

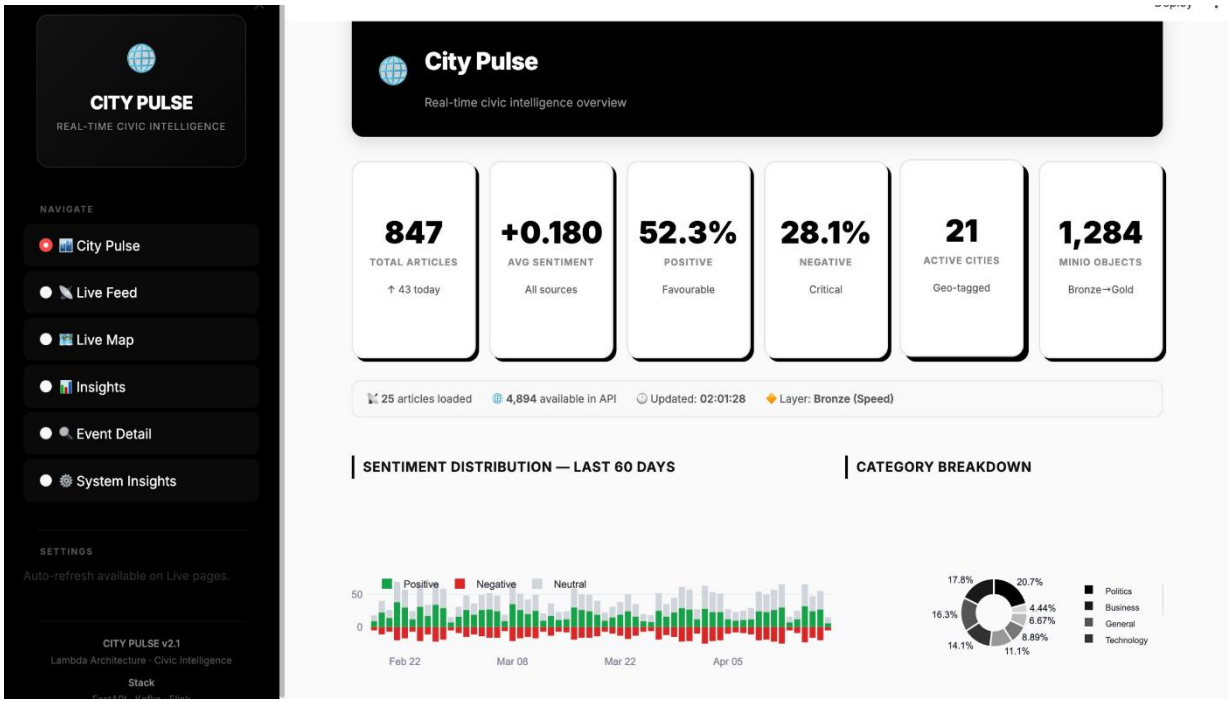


Figure 4.12: The Dashboard display

An interactive map displaying a geospatial analysis of civic data, including event clusters and sentiment by city. Maps display active locations with associated sentiment scores for quick regional insight and comparison

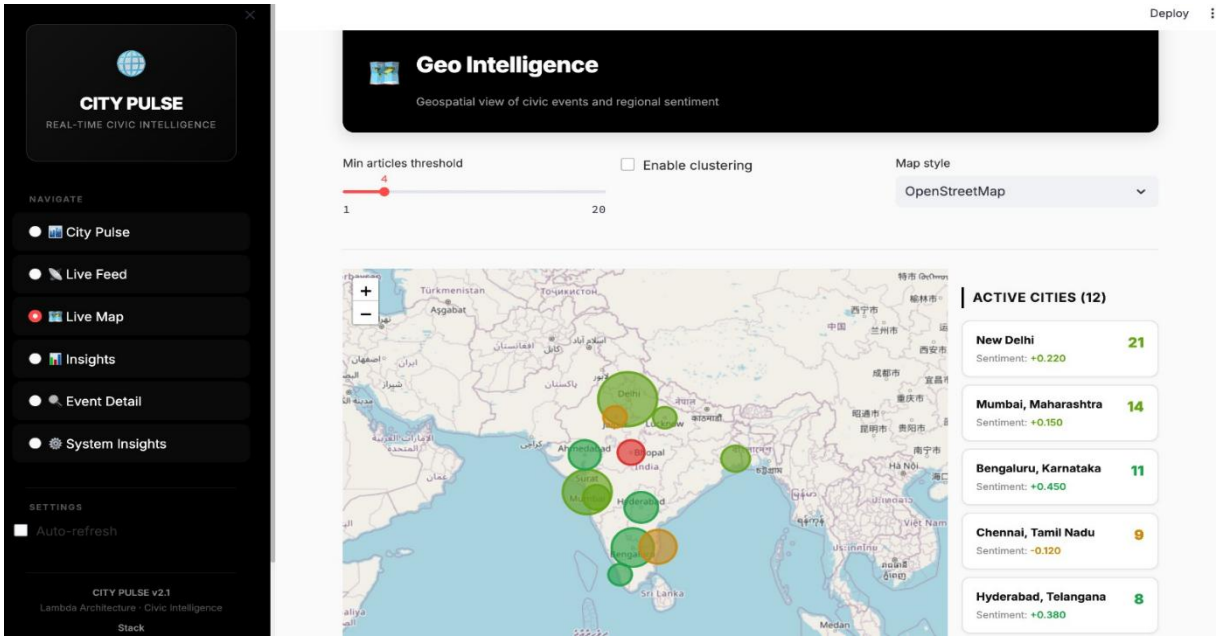


Figure 4.13: Geospatial visualization of civic events and sentiment distribution

This interface provides overall metrics for the total number of articles processed, number of objects processed from Bronze/Silver/Gold layers, and time-based metrics of the articles processed. The interface will show users the daily volume of articles processed each day and also give users an opportunity to investigate trends of multiple analytical dimensions

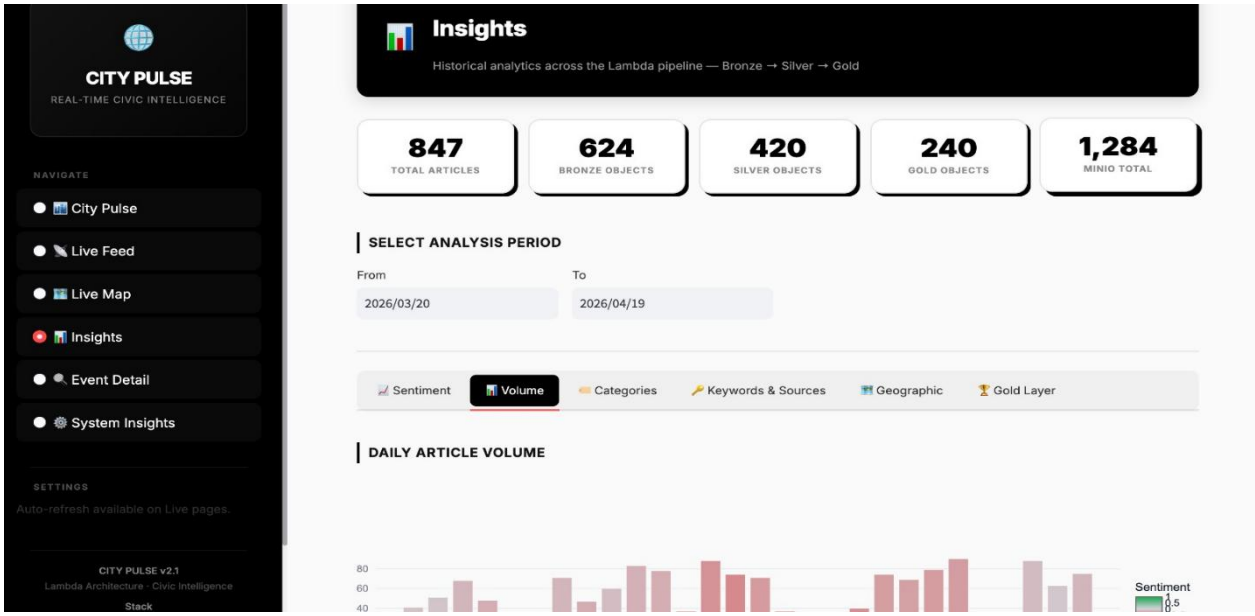


Figure 4.14**:** Insights dashboard showing historical analytics

On this screen, you will be able to see a complete overview of a chosen news article.It will also contain its primary image and organized details like category, source, published date and data pipeline level; for more thorough examination.

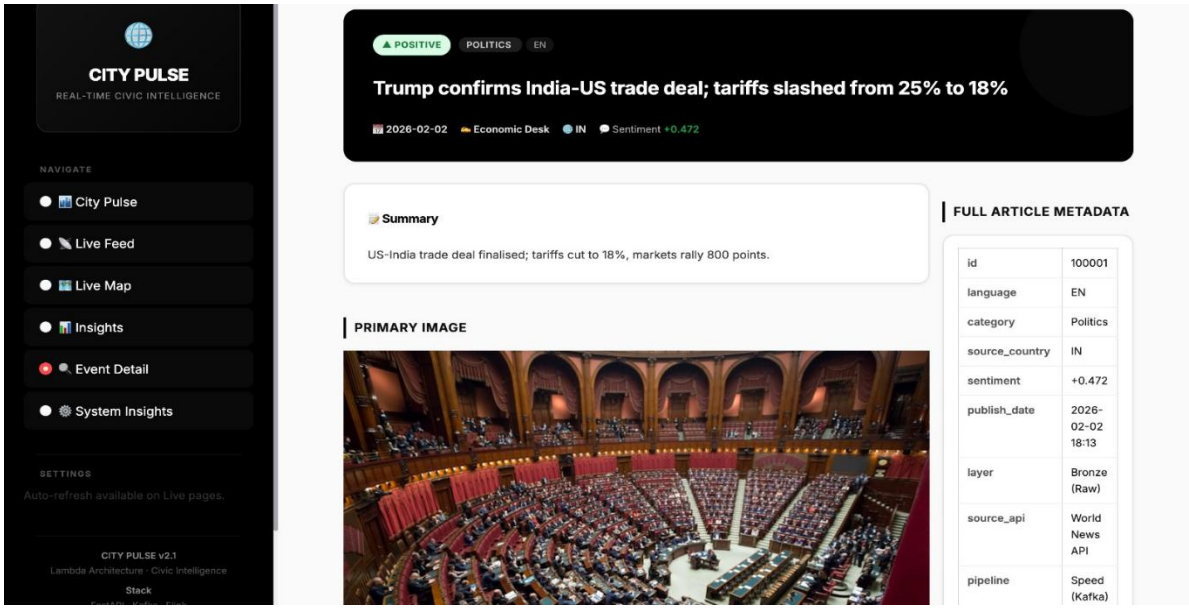


Figure 4.15: Detailed event view displaying news article content and metadata information

System	Processing Approach	Real-Time Capability	Batch / Historical Analytics	Storage Model	Latency Profile	Civic Semantic Understanding	User-Facing Interface
Farajidavar et al. [6]	CNN + CRF on Twitter text streams	Near real-time via Streaming API	Not supported	No persistent storage	Not reported	Multi-category event classification	Not provided
TwitterSensing [7]	Keyword scoring on live Twitter feeds	Continuous, stateless online processing	Not supported	Transient in-memory only	Not reported	Numerical event relevance scoring	Not provided
Yousfi et al. [4]	Lambda-based multimodal batch + stream fusion	Speed layer uses batch-trained models	Hadoop batch layer; traffic domain only	Hadoop HDFS; no tiered structure	Batch-dominant; minutes	Traffic congestion detection only	Not described
Demirezen & Navruz [3]	Lambda architecture on aviation surveillance data	Speed layer for consistency measurement only	Empirical batch-layer accuracy validation	Relational DB; aviation records	Batch range; minutes	No semantic classification	Not provided
Proposed Framework	Hybrid Lambda with Kafka, Flink, real-time consumer	Dedicated speed layer; ~100 ms end-to-end	Flink 5-min tumbling windows; full enrichment pipeline	3-tier Lakehouse (Bronze / Silver / Gold)	~100 ms (speed); 5 min (batch)	Type + severity + location classification	Streamlit dashboard with FastAPI endpoint

Table 2 : Comparative Analysis Of The Proposed Framework With Related Framework

4.3 Performance Evaluation Parameters

To assess the effectiveness of the proposed real-time and batch news data pipeline, the following parameters were used:

1. Latency (Response Time):

Time taken from news ingestion (World News API → Kafka → MongoDB/MinIO) to being available for queries or dashboards. Indicates the system's efficiency in real-time and batch processing.

2. Throughput:

Number of articles processed per second/minute in the Speed Layer and Batch Layer. Evaluates system capacity under varying loads.

3. Scalability

Ability of the system to handle increasing data volumes and concurrent queries without significant performance degradation.

4. User Satisfaction (Qualitative Feedback)

Measures usability and clarity of the dashboard and search results.Collected through user testing, surveys, and feedback on real-time alerts and historical insights

Chapter 5

Conclusion

5.1 Conclusion

This project presents a comprehensive demonstration of how a high-performance, dual-layer news analytics pipeline can be effectively designed to handle the challenges associated with modern, high-velocity data streams. In today's digital ecosystem, where massive volumes of information are continuously generated, the need for systems that can process, analyze, and deliver insights in both real time and at scale has become increasingly critical. The architecture implemented in this work directly addresses these requirements by combining reliable data ingestion mechanisms with advanced processing capabilities.

At the core of the system, Apache Kafka is utilized to ensure resilient and fault-tolerant data ingestion. Its distributed nature allows the pipeline to handle continuous streams of incoming news data without compromising performance or reliability. Complementing this, Apache Flink is employed to enable sophisticated data processing across both real-time and batch layers. This integration ensures that the system not only reacts instantly to new information but also derives deeper insights through historical data analysis, effectively bridging the gap between immediacy and long-term intelligence.

Another key component of the architecture is the adoption of a Lakehouse model implemented using MinIO. This approach establishes a well-organized data lifecycle, where raw, unprocessed news feeds are systematically transformed into structured, high-quality datasets suitable for analytics. By maintaining clear data lineage and integrity, the system ensures that downstream applications and analyses are both reliable and reproducible.

To make these complex processes accessible and actionable, the project also incorporates a FastAPI-based backend alongside a user-friendly Streamlit dashboard. This combination provides a seamless interface that enables users to interact with the data, explore relationships, and extract meaningful insights without needing deep technical expertise.

Overall, this work demonstrates that a Lambda-inspired architectural approach can effectively overcome the latency and scalability challenges commonly found in traditional news monitoring systems. By integrating modern distributed technologies with a structured data framework, the proposed solution offers a powerful and scalable foundation for data-driven decision-making across domains such as media analysis, financial forecasting, and public policy research.

References

[1] J. Kreps, N. Narkhede, and J. Rao, "Kafka: A distributed messaging system for log processing," in Proc. NetDB Workshop, 2011.

[2] P. Carbone et al., "Apache Flink: Stream and batch processing in a single engine," IEEE Data Engineering Bulletin, vol. 38, no. 4, 2015.

[3] M. Z. Demirezen and B. Navruz, "Performance analysis of Lambda architecture-based big-data systems," Sensors, 2023.

[4] S. Yousfi, A. Roubi, and I. Rida, "Towards a generic multimodal architecture for batch and streaming big data integration," Journal of Computer Science, 2019.

[5] T. Komamizu, Y. Amagasa, and H. Kitagawa, "StreamOLAP: Real-time analysis of smart city data," in Proc. IEEE HPCC-SmartCity-DSS, 2016.

[6] N. Farajidavar, T. de Campos, and J. Sherrat, "A deep multi-view learning framework for city event extraction from Twitter data streams," arXiv, 2017.

[7] R. Costa, J. Rodrigues, and J. Neto, "TwitterSensing: An event-based approach for wireless sensor networks optimization using social media," Sensors, 2018.

[8] T. Akidau et al., "Dataflow model: A practical approach to balancing correctness, latency, and cost," Proc. VLDB, 2015.

[9] M. Armbrust et al., "Lakehouse: A new generation of open platforms that unify data warehousing and advanced analytics," Proc. CIDR, 2021.

[10] N. Marz and J. Warren, Big Data: Principles and Best Practices of Scalable Real-Time Data Systems, 2015.

[11] J. Kreps, "Questioning the Lambda Architecture," Confluent Blog, 2014.

[12] T. Akidau, S. Chernyak, and R. Lax, Streaming Systems: The What, Where, When, and How of Large-Scale Data Processing.

[13] M. Zaharia et al., "Discretized Streams: Fault-Tolerant Streaming Computation at Scale," Proceedings of the 24th ACM Symposium on Operating Systems Principles (SOSP), 2013.

[14] G. DeCandia et al., "Dynamo: Amazon's Highly Available Key-Value Store," Proceedings of the 21st ACM Symposium on Operating Systems Principles, 2007.

[15] M. Stonebraker et al., "End of an Architectural Era: It's Time for a Complete Rewrite," Proceedings of the VLDB Endowment, vol. 7, no. 10, 2014.

[16] R. Buyya, S. Srirama, and V. Casale, "Fog and Edge Computing: Principles and Paradigms for Internet of Things Applications," Journal of Systems Architecture, 2019.

[17] H. Gupta, A. Vahid Dastjerdi, S. K. Ghosh, and R. Buyya, "iFogSim: A Toolkit for Modeling and Simulation of Resource Management Techniques in IoT, Edge, and Fog Computing Environments," Software: Practice and Experience, 2017.

[18] J. Manyika et al., "Smart Cities: Digital Solutions for a More Livable Future," McKinsey Global Institute Report, 2018.

[19] B. Babcock et al., "Models and Issues in Data Stream Systems," Proceedings of the 21st ACM SIGMOD-SIGACT-SIGART Symposium on Principles of Database Systems, 2002.

[20] A. Lakshman and P. Malik, "Cassandra: A Decentralized Structured Storage System," ACM SIGOPS Operating Systems Review, vol. 44, no. 2, 2010.

Acknowledgement

We wish to express our sincere gratitude to **Dr. Sanjay U. Bokade, Principal** and **Dr. Jyoti Deshmukh, H.O.D.** of Department of Artificial Intelligence and Data Science of Rajiv Gandhi Institute of Technology for providing us an opportunity to do our project work on **"(**An Agentic Framework for Real-Time Civic Intelligence**)"**.

This project bears on imprint of many peoples. We sincerely thank our project guide Dr.**Nilesh Bhelkar** for his/her guidance and encouragement in carrying out this synopsis work.

Finally, we would like to thank our colleagues and friends who helped us in completing project work successfully

1. Vansh Bohra

2. Disha Jain

3. Drashti Panchal

4.Sakshi Sanap

6% Overall Similarity

The combined total of all matches, including overlapping sources, for each database.

Filtered from the Report

- Bibliography
- Cited Text

Exclusions

- 15 Excluded Matches

Match Groups

32 Not Cited or Quoted 6%
Matches with neither in-text citation nor quotation marks

0 Missing Quotations 0%
Matches that are still very similar to source material

0 Missing Citation 0%
Matches that have quotation marks, but no in-text citation

0 Cited and Quoted 0%
Matches with in-text citation present, but no quotation marks

Top Sources

3% Internet sources
1% Publications
6% Submitted works (Student Papers)

Integrity Flags

0 Integrity Flags for Review

No suspicious text manipulations found.

Our system's algorithms look deeply at a document for any inconsistencies that would set it apart from a normal submission. If we notice something strange, we flag it for you to review.

A Flag is not necessarily an indicator of a problem. However, we'd recommend you focus your attention there for further review.